Title	**Product Return Prediction in E-Commerce Platforms**
Harvard reference	Bivera, J. and Mohan, C.V. (2025). *Product Return Prediction in E-Commerce Platforms*. International Research Journal on Advanced Engineering and Management, 3(12), pp. 3292–3299.
Link	https://www.researchgate.net/publication/399110482_Product_Return_Prediction_in_E-Commerce_Platforms
Modelling	The paper does not clearly mention if the author build together a combination of models into one tested model but it mentions combining models to make return risk predictions. 1. Deep Neural Network was used to review whole backets to make cart- level return predictions and Gradient Boosting Model used to review individual items to score their returns. The results where combined together to get a final prediction. 2. Stacking, Voting and other several models each make a prediction, then their guesses are combined. The document says this approach reached 86% accuracy. 3. Created a hybrid model by combining Convolutional Neural Network (CNN) and Long short-term memory to review 34000 amazon reviews by having CNN save and track customer words and LSTM tracking the sequence of the words to get final sentiment score. They used Adam optimser to train the model to achieve peak performance of 97.67%. 4. K-Means and K-Prototypes was used sort customers into groups with based on the similarity in shopping patterns. 5. HyGraph, LoGraph, and HyperGo were used to look for connections between customer data for example, whether similar customers returned similar items. Additional models used are Digital touch latency (DTL) which predicted returns based on customers behviour on the website and Sentiment drift for reviwing in customers reviews where more negative or positive. 6. To gain a deeper understanding of customer behaviour a Vector embedding model was made using hybrid prepresentation approach of different mathematical modelling techniques such as natural language processing (NLP), behaviour and size profiling and matrix factorisation.

Preprocessing techniques	To create a structured dataset for the predictive models the dataset was 1. cleaned and prepared using statistical imputation techniques to fill missing values. 2. Categorical variables where converted to numerical form. 3. Continious varaibles where normalised to consistent scale, review text where cleaned and tokenised then converted to sentiment scores (short phrases) the computer can process. 4. Numerical data of different ranges was normalised to scale the computer understands using standard 0 to 1 range.
Datasets used	The paper reviews using real transaction data from fashion retailers, 34,000 Amazon reviews for CNN-LSTM training, and various proprietary e-commerce datasets.
Key takeaways	The Hybrid Models of combining DNN for cart-level and GBM for item-level prediction improved accuracy. I could use a similar architecture to maximize prediction accuracy,by fusing deep learning with gradient boosting to analyze both customer text and shopping behavior. I can use an LSTM network to capture chronological patterns in customer reviews, combined with a Deep Neural Network (DNN) to map real-time product switches within active shopping carts. Finally, these outputs will be fed into XGBoost to uncover hidden correlations between cart movements and past feedback, allowing us to predict and mitigate customer returns before checkout. Then training the model with adams optimiser to imrpove performance.

Title	**Analyzing Customer Behavior Patterns & Predicting Online Product Return Intentions: A Data Mining Approach**
Harvard reference	Sharma, N. (2024). *Analyzing Customer Behavior Patterns & Predicting Online Product Return Intentions: A Data Mining Approach*. Master Thesis. Rhine-Waal University of Applied Sciences.

Link	https://opus4.kobv.de/opus4-rhein-waal/frontdoor/deliver/index/docId/1924/file/Neetu_Sharma_Master_thesis.pdf
Modelling	1. Random Forest was used to identifies customer behavior patterns and traits that directly drive product returns by finding features that influence customer behaviour patterns to return. The selection of these features used Random fForest feature selection technique. 2. Seven classification models (Logistic Regression, Decision Tree, Random Forest, AdaBoost, XGBoost, Naive Bayes, and K-Nearest Neighbors) where used to predicts whether a specific customer will return a product, while comparing all seven models to find the top performer.4. Confusion Matrix was used to measure the accuracy of the classification models to determine the most effective model. XGBoost performed best with 76.9% accuracy. 5. Logistic Regression paired with a tuned classification threshold of 0.5 to groups customers into high return risk (probability scores closer to 1) and low return risk (probability scores close to 0) categories based on their online purchasing behavior was used.
Preprocessing techniques	1. Handle missing data: missing values and null entries were removed. The delivery_date column had 9,318 missing values, while user_dob had 8,725. After removing incomplete records, 82,749 records remained. 2. Create derived features: new variables were calculated from existing data: customer age, delivery time, product return rate, and customer return rate. 3. Clean the data: The dataset was checked for outliers and invalid values. Some negative values, included in delivery time were removed. The unwanted item-size and item-color values were also removed.4. Encode categorical data: categories such as user_title, item_size, item_color, and user_state were converted into numerical values so the models could process them easily without difficulty.
Datasets used	A public Kaggle dataset: "e-commerce data germany" (100,000 records initially, 82,749 after cleaning). Contains order details, user demographics (age, gender, state, registration date), delivery information and item attributes (size, colour, brand, price). Return rate: 50.57% (balanced classes)

Key takeaways	Combining Classification threshold to group customers into 2 groups based on the probability score from logisitc regression which predicted the dependent features like return wusing statisitical probability techniques. This combined approach is suitable for Stadioalot's prediction of customer return. Since XGBoost model had the highest (76.9%) accuracy score during its performances, it is considerable to predict returns using XGBoost and Classification threshold of 0.5 because XGBoost outputs numerical values (probability between 0 and 1) like logistic regression, the combination could potentially give better predictions.

Title	Prediction of Order Returns of an Online Clothing Retailer
Harvard reference	Bothe, A.F. (2020). Order Returns Prediction Dataset. GitHub. Available at: https://github.com/AnFrBo/order_returns_prediction [Accessed 28 September 2026].
Link	https://github.com/AnFrBo/order_returns_prediction
Modelling	1.The code loads the cleaned known dataset, selects the modelling variables, replaces missing age values with -99 and removes order_item_id from the predictors while retaining it separately. 2. It then splits the data into stratified training and test sets, using 80% for training and 20% for testing,and converts return into a factor for classification. A probability-based Random Forest model is tuned by grid search over mtry, sampsize and ntree using stratified five-fold cross-validation, producing an untuned model and several tuned versions. 3. The code also trains four XGBoost models, each model is evaluated on the test set using AUC and the predicted probabilities from the four Random Forest and four XGBoost models are averaged to form an ensemble, which is also evaluated using AUC. 4. Finally, the same preprocessing is applied to unknown_clean.csv where the ensemble predicts return probabilities for its records and the item IDs and probabilities are saved to prediction_final.csv.

Preprocessing techniques	1. The raw cvs file was loaded for cleaning onto R script. 2. Date conversions where made on date columns from string to date objects for columns like order_date and user_reg_date. 3. New features where created to add more variables in columns such as children size flag, delivery flag, waiting time flag, low price flag, age flag and gender flag. The reason was to add useful details that will give more insight on predicting items to be returned as the original dataset lacks useful features. 4. Missing values handling by imputing blanks with -99 as a sentinel balue distinct from 0 and 1, to allow the model to treat it as missing values. 5. Feature engineering included reducing dimentionality of features with no information (leakage). 6 The author used feature tuning to test different thresholds before applying feature selections to make sure only values giving useful signals are kepts as variables.
Datasets used	Real-world data provided by an online clothing retailer, made available through a Kaggle challenge was used. The dataset was artificially balanced to a 1:1 ratio between returns and non-returns. It contains order-level information including customer demographics (age, gender, title), product attributes (item size, colour, brand, price), and transaction details (order date, delivery date, user registration date). The known dataset (training) includes the return target; the unknown dataset (test) excludes it. Both are publicly accessible via the GitHub repository.
Key takeaways	1.An R script was utilized to implement a combined ensemble approach featuring XGBoost and Random Forest models, as specified in the requirements.txt file. Combining these algorithms proves more effective than relying on a single model. The performance of this hybrid model was evaluated using the Area Under the Curve (AUC). Because return prediction datasets are inherently imbalanced, complementing AUC with Precision, Accuracy, and F1-score ensures a more comprehensive and robust evaluation. 2. Empirical threshold tuning improves feature quality: The author tested four thresholds for lowprice (14.45, 16.9, 19, 23.9) and chose 23.9 because it gave 11,500 observations with 70% not returned a meaningful signal with enough sample size. I should apply the same empirical tuning to Stadioalot's continuous features like (item_price, delivery_time).

Name	BADS_WS1920_unknown.csv and BADS_WS1920_known.csv

Harvard reference	Bothe, A.F. (2020). Order Returns Prediction Dataset. GitHub. Available at: https://github.com/AnFrBo/order_returns_prediction [Accessed 28 September 2026].
Link	https://github.com/AnFrBo/order_returns_prediction
Number of instances	BADS_WS1920_unknown.csv (Testing dataset split) with 100 000 rows and 14 columns columns.BADS_WS1920_known.csv (Training dataset split) with 50 000 rows and 13 co columns. No mention as to the proportion of split by the author, but I would assume appro test ratio.
Number of feature columns	both datatsets have 13 feature columns
Number of classes (if applicable)	2 classes 0= not returned, 1= returned
Target range	0-1 return binary
(if applicable)	

Origin	The original real-world dataset was sourced from an online retailer via Kaggle. To addres issues, the author artificially balanced the dataset to a 1:1 ratio between returns and non-r subsequently split into training and testing datasets, both of which are publicly hosted on repository.